

Homework 05 Solutions

Prerith Arunkumar (prerith2)

K-nearest neighbors as a local average: its bias–variance trade-off in k , a comparison with lasso under independent and correlated designs, observed versus latent dimension, and a hand-written 5NN digit classifier.

Use of AI tools. I used Claude (Claude Code) to draft the R code and exposition and to run the simulations. Every number below comes from the code output shown in this report, and I checked the numbers quoted in the text against it. Question 5 was, as the assignment allows, written with AI help.

```
library(glmnet)

f <- function(X) 0.5 * X[, 1] + sin(X[, 2]) - 0.3 * X[, 3]^2
x0 <- c(0.5, 0.7, 1)

# Equal-weight KNN regression; returns an (n_test x length(ks)) matrix.
# Squared Euclidean distances via the expansion |a - b|^2 = |a|^2 + |b|^2 - 2 a'b.
knn_reg <- function(Xtr, ytr, Xte, ks) {
  Xte <- matrix(Xte, ncol = ncol(Xtr))
  d2 <- outer(rowSums(Xte^2), rowSums(Xtr^2), "+") - 2 * Xte %*% t(Xtr)
  ord <- matrix(t(apply(d2, 1, order)), nrow = nrow(Xte))
  matrix(sapply(ks, function(k) rowMeans(matrix(ytr[ord[, 1:k, drop = FALSE]], nrow = nrow(Xte)))),
        nrow = nrow(Xte))
}
```

Question 1: Bias and variance of KNN

(a) $f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 = 0.25 + 0.6442 - 0.3 = 0.5942$. Seed 432.

```
ks <- seq(1, 29, 2)
f0 <- f(matrix(x0, 1))
gen1 <- function(n) { X <- matrix(rnorm(n * 3), n); list(X = X, y = f(X) + rnorm(n, sd = 0.1)) }

set.seed(432)
d <- gen1(400)
round(rbind(k = ks, pred = knn_reg(d$X, d$y, x0, ks)[1, ]), 3)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]	[,11]
k	1.000	3.000	5.000	7.000	9.000	11.000	13.000	15.000	17.000	19.000	21.000
pred	0.752	0.752	0.544	0.459	0.482	0.409	0.454	0.508	0.545	0.517	0.497
	[,12]	[,13]	[,14]	[,15]							
k	23.000	25.000	27.000	29.000							
pred	0.548	0.546	0.476	0.456							

(b) Seed 432. Each row of P is one training set, reused for all 15 values of k .

```

set.seed(432)
P <- t(replicate(200, { d <- gen1(400); knn_reg(d$X, d$y, x0, ks)[1, ] }))
bias2 <- (colMeans(P) - f0)^2
vars <- colMeans(sweep(P, 2, colMeans(P))^2)          # divisor 200
mse <- colMeans((P - f0)^2)
c(dim = dim(P), max_abs_gap = max(abs(mse - bias2 - vars)))

```

```

      dim1      dim2 max_abs_gap
2.000000e+02 1.500000e+01 7.459311e-17

```

```

round(rbind(k = ks, bias2, vars, mse), 4)

```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]
k	1.0000	3.0000	5.0000	7.0000	9.0000	11.0000	13.0000	15.0000	17.0000
bias2	0.0003	0.0021	0.0020	0.0029	0.0033	0.0043	0.0051	0.0057	0.0070
vars	0.0421	0.0190	0.0123	0.0106	0.0088	0.0084	0.0083	0.0077	0.0072
mse	0.0424	0.0212	0.0143	0.0135	0.0121	0.0126	0.0134	0.0134	0.0143

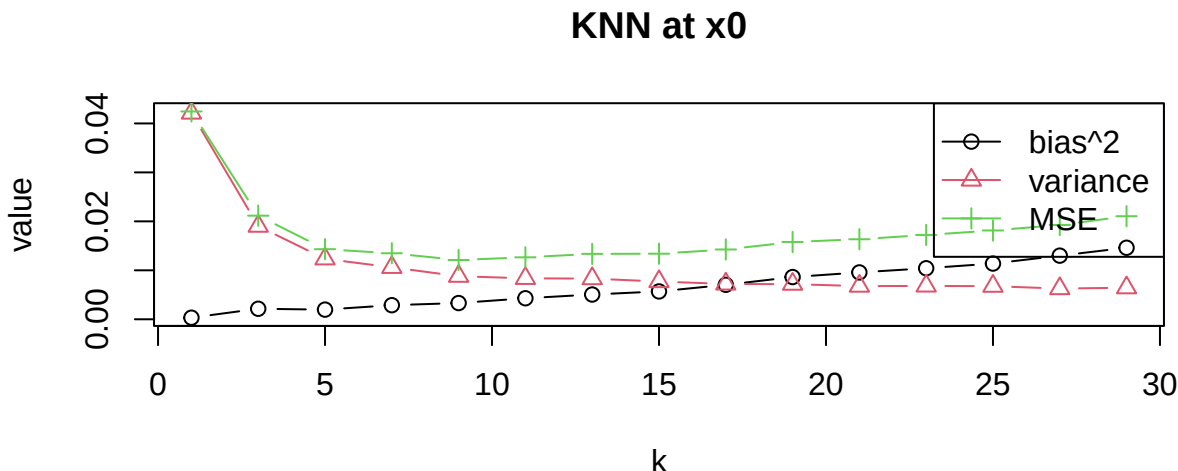
	[,10]	[,11]	[,12]	[,13]	[,14]	[,15]
k	19.0000	21.0000	23.0000	25.0000	27.0000	29.0000
bias2	0.0086	0.0096	0.0104	0.0114	0.0130	0.0146
vars	0.0072	0.0068	0.0068	0.0068	0.0063	0.0064
mse	0.0158	0.0163	0.0172	0.0181	0.0193	0.0210

The identity $MSE = bias^2 + variance$ holds to machine precision, as it must with divisor 200: it is an algebraic identity for the empirical moments, not an approximation.

```

matplot(ks, cbind(bias2, vars, mse), type = "b", pch = 1:3, lty = 1, col = 1:3,
        xlab = "k", ylab = "value", main = "KNN at x0")
legend("topright", c("bias^2", "variance", "MSE"), col = 1:3, pch = 1:3, lty = 1)

```



Variance drops steeply for small k and then levels off. It is much larger than the response-noise share $0.1^2/k$ (only 0.0003 at $k = 29$), so it is dominated by *where* the neighbors land: each training set puts different points near $\mathbf{x}_0$, and f varies over them. Averaging more neighbors smooths that out, but with diminishing returns. Squared bias grows with k because the neighborhood widens and starts averaging f over points where it differs from $f(\mathbf{x}_0)$; f is concave in x_2 (near 0.7) and x_3 , and $\mathbf{x}_0$ is off-center ($\|\mathbf{x}_0\|^2 = 1.74$), so wider neighborhoods also lean toward

the denser region nearer the origin, and both effects pull the average away. MSE is U-shaped, bottoming out near $k \approx 9$. This is the same trade-off as ridge and lasso with k playing the role of λ : larger k (like larger λ) means more smoothing, lower variance, and more bias. The difference is the kind of bias. Ridge and lasso shrink global coefficients toward zero, whereas KNN's bias is local and comes from averaging f over a neighborhood, so it depends on the curvature of f near $\mathbf{x}_0$ and on how densely the data cover that point.

Question 2: KNN versus lasso

The equicorrelated design uses the exact one-factor representation $\mathbf{x}_i = \sqrt{1-\rho}\mathbf{z}_i + \sqrt{\rho}w_i\mathbf{1}$ with $\mathbf{z}_i \sim N(\mathbf{0}, I_{30})$ and $w_i \sim N(0,1)$, which gives unit variances and pairwise covariance ρ . Setting 1 is $\rho = 0$. Standardization uses divisor equal to the sample size of the data it is estimated on, and it is re-estimated inside every fold. `glmnet` with `intercept = FALSE` and `standardize = FALSE` minimizes exactly the stated objective on the transformed data; the intercept is the training mean of y .

```
gen2 <- function(n, rho) {
  X <- sqrt(1 - rho) * matrix(rnorm(n * 30), n) + sqrt(rho) * rnorm(n)
  list(X = X, y = f(X) + rnorm(n, sd = 0.1))
}
lams <- 10^seq(0, -4, length.out = 41)
lasso_fit <- function(X, y) {
  m <- colMeans(X); s <- sqrt(colMeans(sweep(X, 2, m)^2)); ym <- mean(y)
  tf <- function(Z) sweep(sweep(Z, 2, m), 2, s, "/")
  g <- glmnet(tf(X), y - ym, lambda = lams, standardize = FALSE, intercept = FALSE, thresh = 1e-12)
  list(g = g, pred = function(Z) ym + as.matrix(predict(g, tf(Z))))
}
lasso_cv <- function(X, y) {
  folds <- sample(rep(1:10, length.out = nrow(X)))
  cvm <- rowMeans(sapply(1:10, function(v) {
    fit <- lasso_fit(X[folds != v, ], y[folds != v])
    colMeans((fit$pred(X[folds == v, , drop = FALSE]) - y[folds == v])^2)
  })))
  list(fit = lasso_fit(X, y), j = which.min(cvm))
}
```

(a) Seed 432, set once before both settings.

```
set.seed(432)
res <- t(sapply(c(0, 0.8), function(rho) {
  tr <- gen2(400, rho); te <- gen2(1000, rho)
  cv <- lasso_cv(tr$X, tr$y)
  b <- coef(cv$fit$g)[-1, cv$j]
  c(rho = rho, mse_5NN = mean((knn_reg(tr$X, tr$y, te$X, 5) - te$y)^2),
    mse_lasso = mean((cv$fit$pred(te$X)[, cv$j] - te$y)^2),
    lambda = lams[cv$j], nonzero = sum(b != 0))
})))
knitr::kable(data.frame(setting = 1:2, res), digits = 4)
```

setting	rho	mse_5NN	mse_lasso	lambda	nonzero
1	0.0	0.6020	0.2847	0.0316	4
2	0.8	0.1721	0.2319	0.0200	8

(b) With independent covariates, 5NN is badly hurt: 27 of the 30 directions are pure noise, and they dominate the Euclidean distance, so the five “nearest” neighbors are close in noise coordinates and only weakly related in

(x_1, x_2, x_3) . Its test MSE is far larger than the lasso's. Under $\rho = 0.8$ the 30 covariates mostly move together along the common factor w_i , so the cloud is close to one-dimensional plus a smaller independent part with variance 0.2 per coordinate. Distances then track differences in w_i , which also drives x_1, x_2, x_3 , and neighborhoods become far more informative: 5NN improves by a factor of about 3.5 and overtakes the lasso. Lasso works differently. It fits one coefficient per covariate and can set noise coefficients exactly to zero, so irrelevant covariates are (mostly) removed rather than contributing to a distance. KNN weights every coordinate equally and cannot ignore any of them. Under correlation the lasso keeps more slopes (8 versus 4), but the extra ones are not useful proxies. For Gaussian covariates, Stein's lemma gives the population linear projection of f as $0.5x_1 + E[\cos X_2] x_2 \approx 0.5x_1 + 0.61x_2$ in *both* settings, so once x_1 and x_2 are in, no other covariate (including x_3) carries linear information. The extra slopes are tiny noise variables admitted at a small cross-validated λ , which is the lasso's known tendency with correlated designs. Lasso never selects x_3 in either setting.

(c) The lasso is misspecified: it can capture the linear part $0.5x_1$ and the roughly linear part of $\sin(x_2)$, but $-0.3x_3^2$ has zero linear correlation with x_3 (and with every other covariate), so that part is left in the residual for every λ . Its test MSE therefore has a floor of about 0.25 (the unexplained parts of $\sin(x_2)$ and $-0.3x_3^2$ plus the noise 0.01), far above 0.01. With 1,000 test points each MSE also carries a standard error of roughly 0.01–0.02, so the lasso's small change between settings is within test-sample noise. KNN has no such floor in principle. Part of KNN's win under correlation is thus “the linear model is wrong”, not only “correlation helps neighborhoods”. The results do not justify general claims. They come from one dataset per setting, one n , one ρ , k fixed at 5, and one response function. Correlation helped KNN here because the shared factor drives the signal covariates. If the correlation lay in directions unrelated to the signal, it could shrink the effective dimension without making neighbors any more alike in f . Likewise, with a truly sparse *linear* f , lasso would win easily in both settings. Which method is better depends on the structure of f and of $\mathbf{X}$.

Question 3: Observed versus latent dimension

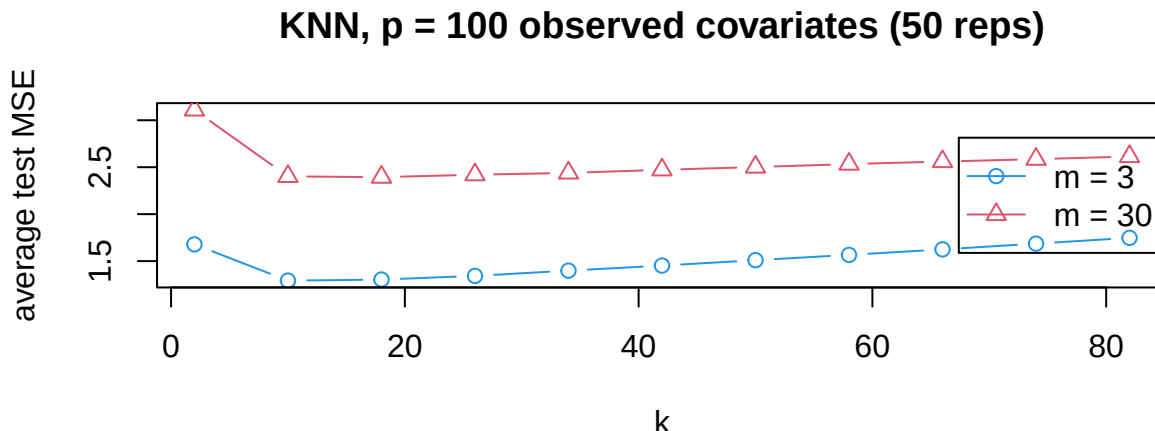
```
ks3 <- seq(2, 82, 8)
gen3 <- function(n, A) {
  Z <- matrix(rnorm(n * ncol(A)), n)
  list(X = Z %*% t(A) + matrix(rnorm(n * 100), n), y = Z[, 1] + Z[, 2] + rnorm(n))
}
one3 <- function(m) {
  A <- matrix(runif(100 * m, -2, 2), 100); tr <- gen3(200, A); te <- gen3(200, A)
  colMeans((knn_reg(tr$X, tr$y, te$X, ks3) - te$y)^2)
}
set.seed(432) # (a) one dataset per m
round(rbind(k = ks3, m3 = one3(3), m30 = one3(30)), 3)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]	[,11]
k	2.000	10.000	18.000	26.000	34.000	42.000	50.000	58.000	66.000	74.000	82.000
m3	1.442	1.120	1.186	1.211	1.265	1.335	1.381	1.425	1.479	1.539	1.604
m30	2.994	2.265	2.197	2.240	2.275	2.266	2.288	2.308	2.331	2.341	2.375

```
set.seed(4320) # (b) 50 repetitions
avg <- sapply(c(3, 30), function(m) rowMeans(replicate(50, one3(m))))
round(rbind(k = ks3, m3 = avg[, 1], m30 = avg[, 2]), 3)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]	[,11]
k	2.000	10.000	18.000	26.000	34.000	42.000	50.000	58.000	66.000	74.000	82.000
m3	1.678	1.292	1.303	1.343	1.399	1.452	1.510	1.566	1.625	1.686	1.748
m30	3.110	2.402	2.394	2.420	2.439	2.472	2.502	2.533	2.560	2.585	2.614

```
matplot(ks3, avg, type = "b", pch = 1:2, lty = 1, col = c(4, 2), xlab = "k",
        ylab = "average test MSE", main = "KNN, p = 100 observed covariates (50 reps)")
legend("right", c("m = 3", "m = 30"), col = c(4, 2), pch = 1:2, lty = 1)
```



(c) The response depends only on $Z_1 + Z_2$ in both settings, but KNN never sees $\mathbf{Z}$. It measures distance in $\mathbb{R}^{100}$, where $\|\mathbf{x}_i - \mathbf{x}_j\|^2 = \|\mathbf{A}(\mathbf{z}_i - \mathbf{z}_j)\|^2 + \|\eta_i - \eta_j\|^2 + \text{cross term}$. When $m = 3$, the signal part is a 3-dimensional geometry stretched by $\mathbf{A}$ (entry variance $4/3$, so each latent direction is amplified strongly), and 200 points cover a 3-dimensional space reasonably well. Nearby $\mathbf{x}$'s therefore have nearby $\mathbf{z}$'s, and so nearby $Z_1 + Z_2$. When $m = 30$, the neighbors must be close in 30 latent directions at once, and with only 200 points this is the curse of dimensionality: distances become much less discriminating. Worse, 28 of those 30 directions are irrelevant to y , yet they carry most of the distance. Across 50 repetitions the best average test MSE is 1.29 for $m = 3$ (noise floor 1) versus 2.39 for $m = 30$, a modest improvement over predicting with the mean, whose MSE is $\text{Var}(Y) = 3$.

The noise η_i adds an independent $N(\mathbf{0}, I_{100})$ perturbation in *every* direction, including the $100 - m$ directions orthogonal to the column space of $\mathbf{A}$. So $\mathbf{X}_i$ has a full-rank covariance $\mathbf{A}\mathbf{A}^T + I$ and does not lie exactly on an m -dimensional subspace. It lies *near* one, in the sense that the m signal directions have variance of order $100 \cdot \frac{4}{3}$ against 1 for the noise directions. The noise adds a nearly constant amount, about $2 \cdot 100$ in expectation, to every squared distance, plus random fluctuation that blurs which neighbor is truly closest. The expected signal part of a squared distance is about $2 \cdot m \cdot 100 \cdot \frac{4}{3}$, around 800 for $m = 3$ and 8,000 for $m = 30$, against 200 for the noise. So the gap between the two settings comes from the latent dimension, not from η_i . The noise mostly perturbs the fine ranking of close neighbors, which matters more when m is small and true neighbors really are close. This is why the digits work: 256 pixels, but the images lie near a low-dimensional set.

Question 4: Handwritten digits

The data are the course CSV copies of ElemStatLearn's `zip.train/zip.test` (the same values; first column is the label).

```
rd <- function(f) as.matrix(read.csv(gzfile(file.path("data/knn", f)), header = FALSE))
trn <- rd("zip-train.csv.gz")[1:1000, ]; tst <- rd("zip-test.csv.gz")[1:500, ]
Xtr <- trn[, -1]; ytr <- trn[, 1]; Xte <- tst[, -1]; yte <- tst[, 1]
```

(a) Distances are computed directly for each test image. `order()` is stable, so equal distances keep the smaller training row first. `which.max` returns the first maximum of the vote counts, indexed 0 to 9, so tied votes go to the smallest digit.

```
knn5 <- function(x0, Xtr, ytr, dist = c("euclidean", "manhattan"), k = 5) {
  dist <- match.arg(dist)
  D <- t(Xtr) - x0 # 256 x n_train differences
  d <- if (dist == "euclidean") sqrt(colSums(D^2)) else colSums(abs(D))
  votes <- tabulate(ytr[order(d)[1:k]] + 1, nbins = 10)
  which.max(votes) - 1
}
```

(b)

```
pred <- sapply(c("euclidean", "manhattan"), function(m)
  apply(Xte, 1, knn5, Xtr = Xtr, ytr = ytr, dist = m))
cm <- lapply(1:2, function(j) table(pred = factor(pred[, j], 0:9), true = factor(yte, 0:9)))
colMeans(pred != yte) # test classification error
```

```
euclidean manhattan
0.122      0.138
```

```
cm[[1]]; cm[[2]]
```

```
      true
pred  0   1   2   3   4   5   6   7   8   9
0 118   0   7   1   0   4   2   0   2   0
1   0  66   0   0   2   0   0   0   0   1
2   0   0  39   1   0   0   2   0   0   1
3   0   0   0  23   0   4   0   0   2   0
4   0   0   1   0  27   1   1   1   0   1
5   0   0   0   1   0  13   0   0   0   0
6   0   2   0   0   0   1  41   0   2   0
7   0   0   0   0   1   1   0  34   0   4
8   0   0   3   2   0   2   0   0  37   0
9   1   0   0   1   4   0   0   1   1  41
```

```
      true
pred  0   1   2   3   4   5   6   7   8   9
0 118   0   6   1   0   5   3   0   4   0
1   0  67   2   0   2   0   1   0   0   1
2   0   0  35   0   0   0   2   0   0   0
3   0   0   1  24   0   4   0   0   2   0
4   0   0   1   1  27   1   0   1   0   1
5   0   0   0   1   0  11   0   0   0   0
6   0   1   0   0   0   2  40   0   2   0
7   0   0   2   1   1   0   0  34   0   5
8   0   0   2   0   0   3   0   0  34   0
9   1   0   1   1   4   0   0   1   2  41
```

```
offdiag <- function(t) { t <- unclass(t); diag(t) <- 0
  i <- which(t >= 4, arr.ind = TRUE) # every off-diagonal cell with count >= 4
  d <- data.frame(pred = i[, 1] - 1, true = i[, 2] - 1, n = t[i]); d[order(-d$n), ] }
offdiag(cm[[1]]); offdiag(cm[[2]])
```

```
pred true n
1     0    2 7
```

2	9	4 4
3	0	5 4
4	3	5 4
5	7	9 4

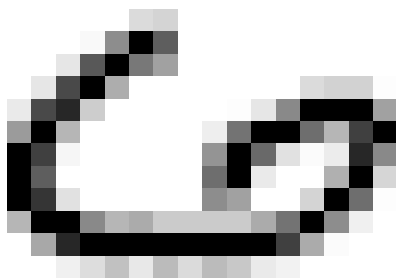
	pred	true	n
1	0	2	6
3	0	5	5
6	7	9	5
2	9	4	4
4	3	5	4
5	0	8	4

Euclidean error is 0.122 and Manhattan error is 0.138. Every off-diagonal cell with count at least 4 is listed above. Counting both directions of a pair, the digits most often confused with one another are 0 and 2 (7 Euclidean, 6 Manhattan), then 3 and 5, 4 and 9, and 7 and 9 (5 each under Euclidean; under Manhattan 7 and 9 reach 6, and 0 and 5 join at 5). Nearly all of these are one-directional: true 2s, 5s and 8s pulled to 0, true 4s to 9, true 9s to 7. These pairs share most of their strokes: closed loops (0, 2 with a looped base, 5), and the open 4/9/7 shapes. The digit 0 acts as an “attractor”: it is the most common class in the training rows, and heavy, round images sit close to many 0s.

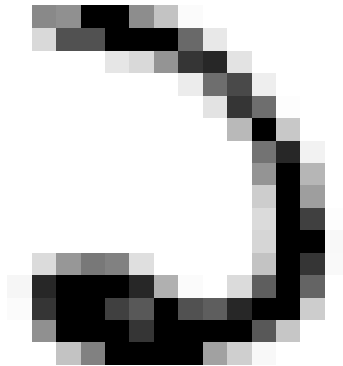
(c) Three misclassified test images under Euclidean distance:

```
wrong <- which(pred[, 1] != yte)
op <- par(mfrow = c(1, 3), mar = c(0.5, 0.5, 2, 0.5))
for (i in wrong[1:3]) {
  image(t(matrix(Xte[i, ], 16, 16, byrow = TRUE))[, 16:1], col = gray.colors(64, 1, 0),
        axes = FALSE, main = sprintf("true %d, pred %d", yte[i], pred[i, 1]))
}
```

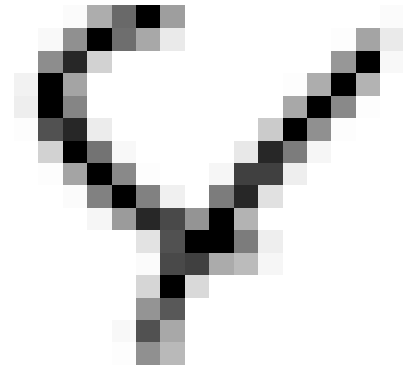
true 6, pred 0



true 2, pred 0



true 4, pred 7



```
par(op)
both_wrong <- sum(pred[, 1] != yte & pred[, 2] != yte)
c(euclid_only = sum(pred[, 1] != yte & pred[, 2] == yte),
  manhattan_only = sum(pred[, 2] != yte & pred[, 1] == yte), both = both_wrong,
  disagree = sum(pred[, 1] != pred[, 2]))
```

euclid_only	manhattan_only	both	disagree
3	11	58	22

The displayed errors are visually understandable. The 6 is written flat, so its loop and tail trace an oval like a 0. The 2 has a thick, filled base that covers the pixels of a 0's lower half. The 4 has an open top, and its two strokes

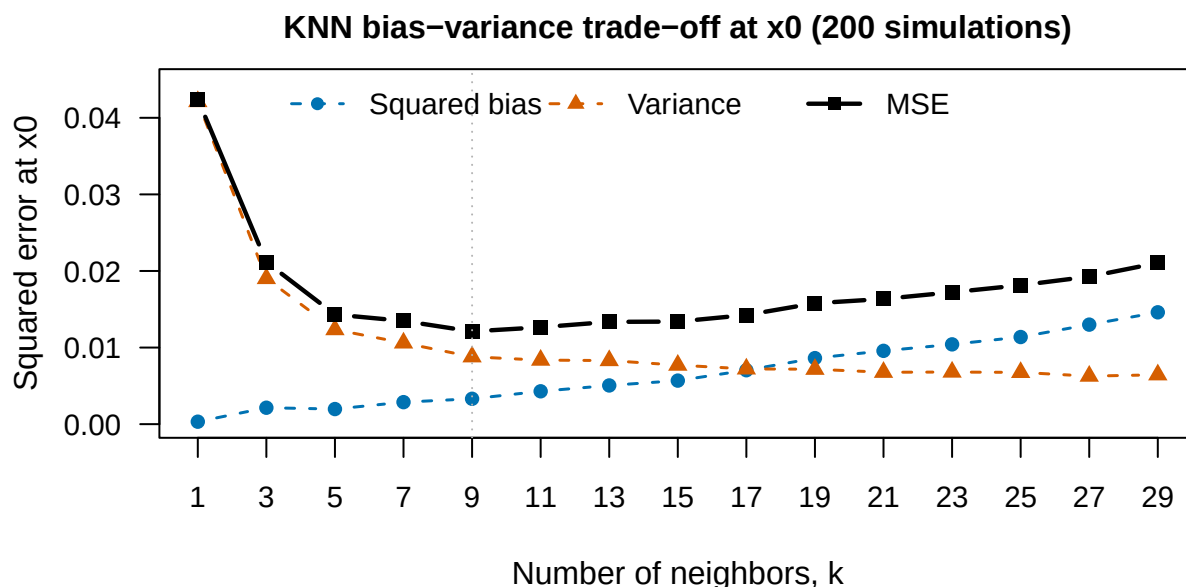
form a 7-like diagonal. The overall results are close (errors 0.122 versus 0.138), and most wrong images are wrong under both distances. Still, the two classifiers disagree on 22 images. Three images are wrong only under Euclidean distance and 11 only under Manhattan distance. For example, test image 13 (a true 2) is predicted 0 by Euclidean and 3 by Manhattan, and image 381 (a true 3) is predicted 8 by Euclidean but correctly by Manhattan. So yes: similar totals can hide different mistakes. Euclidean distance squares pixel differences, so a few large mismatches (a stroke shifted by one pixel) dominate. Manhattan distance counts all differences linearly, which changes the neighbor ranking mainly for borderline images. Similar total error is therefore compatible with different error sets.

Question 5: A plot-formatting skill

(a) I wrote the skill below (with AI help) and installed it in this repository at `.claude/skills/stat432-plot-format/SKILL.md`.

(b) After installing the skill, I asked the agent to apply it explicitly to the Question 1 plot. The original is above; the revised version follows.

```
cols <- c("#0072B2", "#D55E00", "#000000") # colorblind-safe
op <- par(mar = c(4.2, 4.5, 2.2, 1), cex.axis = 0.9, cex.lab = 1, las = 1)
matplot(ks, cbind(bias2, vars, mse), type = "b", pch = c(16, 17, 15), lty = c(2, 2, 1),
        lwd = c(1.5, 1.5, 2.2), col = cols, xaxt = "n", ylim = c(0, max(mse) * 1.05),
        xlab = "Number of neighbors, k", ylab = "Squared error at x0",
        main = "KNN bias-variance trade-off at x0 (200 simulations)", cex.main = 1)
axis(1, at = ks)
abline(v = ks[which.min(mse)], col = "grey70", lty = 3)
legend("top", c("Squared bias", "Variance", "MSE"), col = cols, pch = c(16, 17, 15),
      lty = c(2, 2, 1), lwd = c(1.5, 1.5, 2.2), horiz = TRUE, bty = "n", cex = 0.9)
```



```
par(op)
```

Compared with the original, the revision has descriptive axis labels, a title that states what was simulated, every value of k on the x-axis, a y-axis starting at zero, colorblind-safe colors with distinct marker shapes, a heavier line for the MSE (the quantity of interest), a legend that no longer covers the data, and a dotted marker at the MSE-minimizing k .

Assessment (drafted with AI help). Overall the changes are an improvement. The descriptive labels and every evaluated k on the axis make the plot readable without the code, and the heavier MSE line with the reference at

its minimizer points straight to the trade-off. Starting the y-axis at zero is also more honest about how small the squared bias is at small k . Two things are less good: the horizontal legend is cramped at this figure width, and the dotted minimizer line adds a mild emphasis the original did not have. I kept the skill and revised it once, replacing an example that echoed a Question 1 variable name with a generic one so the rules stay portable.

(c) Final SKILL.md:

```
---
name: stat432-plot-format
description: Improve the formatting of a plot in a STAT 432 homework report (R or Python). Use only when told to use "plot-format" on a specific plot. Do not change the data, the computation, or the plot type.
---

# stat432-plot-format

Restyle one existing homework plot. Change presentation only; the plotted numbers must stay identical.

## Rules

1. Margins. Nothing may be clipped: axis labels, tick labels, legend, and title all fully visible. Trim white space (R: set par(mar = ...) and restore it afterwards; Python: fig.tight_layout() or layout).
2. Title. One line, sentence case. Say what is shown and under what setting (for example sample size or number of simulations). Do not repeat the axis labels.
3. Axes. Descriptive labels in words, with units or scale where they apply. Show ticks at the values actually evaluated when there are about 15 or fewer. Start a y-axis at zero for errors, variances, and counts unless the pattern. Keep tick labels horizontal when they fit.
4. Labels and legend. Spell out series names ("Test error", not a raw variable name). Place the legend, data, or label lines directly. Mark a highlighted value (for example a minimizer) with a light reference line.
5. Colors. Use a colorblind-safe palette (Okabe-Ito: #0072B2, #D55E00, #009E73, #CC79A7, #000000). Use a second cue (marker shape or line type) so the plot survives grayscale printing. Use at most two colors.
6. Sizes. Text must be readable at the rendered page size: tick labels about 9 pt, axis labels and titles about 12 pt. Make the key series visibly heavier than supporting ones. Set the figure size so the plot fits a half page.
7. Scope. Report in one or two sentences what changed. Do not add new analysis, smoothing, or annotation. State conclusions.
```

I kept the rules short and limited to things I would otherwise fix by hand every week: labels with units, readable sizes, margins that do not clip, and colors that survive grayscale printing and colorblindness. The rules never mention a specific question, so the skill stays portable. The description requires an explicit request, so the agent will not restyle plots on its own while drafting.